Consider a vocabulary $\mathcal{V} = (A, T, C, G)$ and the corresponding embedding matrix $E = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \\ 1 & 1 \end{bmatrix}$. For example, the $0-th$ row of E is the embedding for $0-th$ token of $\mathcal{V}$ and so on.

The following sequence $X = (G, C, C, T, A, G)$ is to be processed by the encoder layer of the transformer architecture by appropriately adding the positional information using the following encoding scheme

$$P(pos, 2i) = sin\left(\frac{pos}{10^{(2i/dmodel)}}\right)$$

$$P(pos, 2i+1) = cos\left(\frac{pos}{10^{(2i/dmodel)}}\right)$$

Assume the index for the position starts from zero. Add positional information to all the tokens in the sequence. Let the resultant sequence be denoted by $H = E(X) + P$, (H will be used in the subsequent questions, please write it down). Here, $E(X)$ denotes the embedding for each token in X

What is the sum of the elements in the embedding vector P_5?

Pos-	0	1	2	3	4	5	
P	G	C	C	T	A	G	i=
PE	0	0.84	0.91	0.14	-0.75	-0.95	0
	1	0.54	-0.41	-0.98	-0.65	0.28	0

$PE(G) = \sin\left(\frac{0}{1}\right) \quad \cos(0)$

$PE_1(C) = \sin\left(\frac{1}{10^{0/2}}\right) = \sin(1) \quad \cos(1)$

$= 0.84, \quad 0.54$

$PE_2(C) = \sin(2), \cos(2)$

$0.91, \quad -0.41$

$PE_3(T) = \sin(3), \cos(3)$

$0.14, \quad -0.98$

$PE_4(A) = \sin(4), \cos(4)$

$-0.75 \quad -0.65$

$PE_5(G) = \sin(5) \quad \cos(5)$

$-0.95 \quad 0.28$

Pos-	0	1	2	3	4	5	
P	G	C	C	T	A	G	i=
PE	0	0.84	0.91	0.14	-0.75	-0.95	0
	1	0.54	-0.41	-0.98	-0.65	0.28	0

$\therefore P_5 = -0.95 \quad 0.28 \qquad \therefore Sum = -0.67$

H (row form) $= E(x) + P =$

1	2	
1.84	-0.46	h_0
1.91	-1.41	h_1
0.14	0.02	h_2
0.25	-0.65	h_3 *
0.05	1.28	h_4
		h_5

What is the sum of the elements in the $3-rd$ token, $h_3 \in H$

$h_3 = 0.14 \quad 0.02$

Sum = 0.16

The bottommost encoder layer of the transformer contains the following parameter matrices

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \quad W_V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad W_O = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

given that the embeddings are row vectors, compute the attention score to produce the new representation for the token (T) at the 3-rd position in the sequence X.

$H =$

1	2
1.84	-0.46
1.91	-1.41
0.14	0.02
0.25	-0.65
0.05	1.28

What is the **un-normalized** attention weight given for the token (A) at the 4th position?

Note: unnormalized here implies the raw score before applying softmax. Avoid scaling the dot-product by $\sqrt{d_k}$

$Q = H \cdot W_Q =$

1	-2
1.84	0.46
1.91	1.41
0.14	-0.02
0.25	0.65
0.05	-1.28

, $K = H \cdot W_K =$

3	0
1.38	0
0.5	0
0.16	0
-0.40	0
1.33	0

$\therefore$ Scores $= Q \cdot K^T =$

	1	-2
G		
C	1.84	0.46
C	1.91	1.41
T	0.14	-0.02
A	0.25	0.65
G	0.05	-1.28

$\cdot \begin{bmatrix} 3 & 1.38 & 0.5 & 0.16 & -0.40 & 1.33 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

G	C	C	T	A	G.
0.42	0.14x 1.38	0.07	0.14x 0.16	0.14 x -0.40	0.14x 1.33

So for T, $\rightarrow$

Unnormalized Score for T (A) $\rightarrow$ $0.14 \times (-0.4)$

$= -0.056$

The bottommost encoder layer of the transformer contains the following parameter matrices

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \quad W_V = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad W_O = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

given that the embeddings are row vectors, compute the attention score to produce the new representation for the token (T) at the 3-rd position in the sequence X.

Which token in the sequence was given the highest score?

G C C T A G.

We got Score $(T) = [0.42 \quad 0.19 \quad 0.07 \quad .0224 \quad -0.056 \quad 0.18]$

$\therefore$ highest Score (0.42) for G.

Consider the following configuration for the Vannila transformer architecture with one encoder layer and one decoder layer.

- Source and target vocabulary size = 100
- maximum sequence length = 32
- length of context window (T) for both encoder and decoder = 32
- number of heads $n_h = 2$
- $dff = 4 * dmodel$
- $dq = dk = dv = \frac{dmodel}{n_h}$

Suppose the number of learnable parameters in the source input embedding layer is 1600, how many parameters are there in the positional embedding layer of the source language? Assume the positional embeddings are learnable.

Embedding layer Size = $d_{model} \times$ Vocab_Size.

$\therefore d_{model} = \frac{1600}{100} = \underline{16}$

$\therefore$ Every token is represented by 16 variables.

Number of Parameters in PE = $n \times d_{model}$.

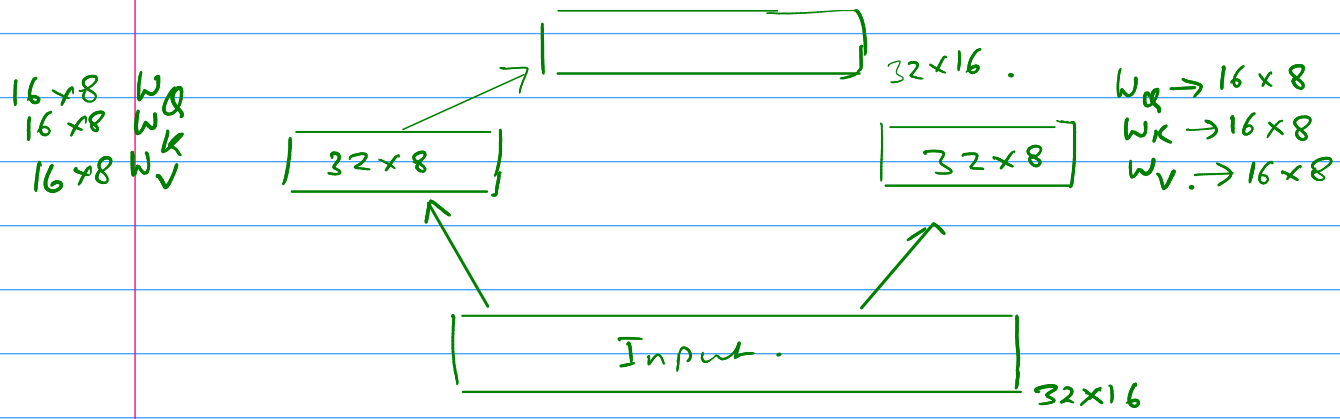
$n \rightarrow$ Sequence length.

$\therefore$ no. of Parameters in PE = $32 \times 16 = \underline{512}$

How many parameters are there in the multi-head attention layer of the encoder (exclude the parameters in the W_O matrix used for linear transformation)?

Means Parameters in W_Q, W_K & W_V.

Input = 32 × 16



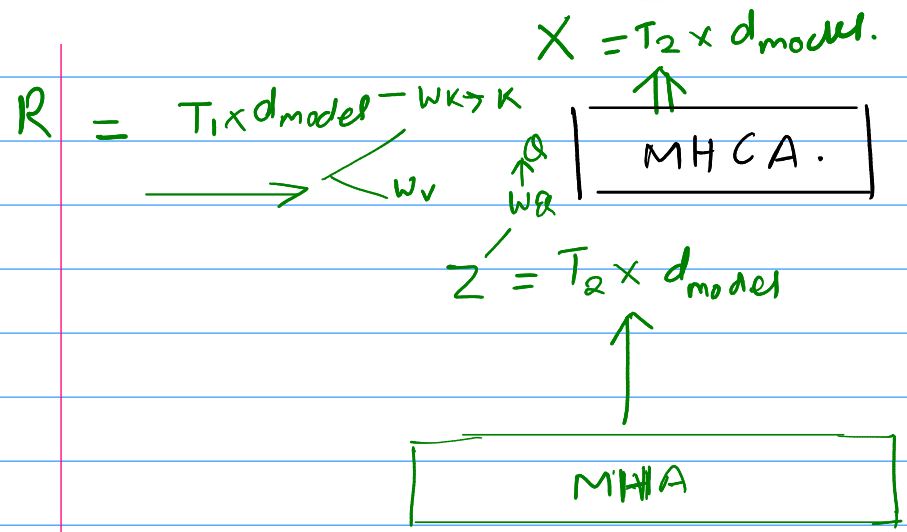
$(16 \times 8 \times 3) \times 2 = 16 \times 16 \times 3$

$= 256 \times 3$

$= 768$

Let $R_E \in \mathbb{R}^{T_1 \times dmodel}$ be the representation from the topmost encoder layer of the encoder and $Z_D \in \mathbb{R}^{T_2 \times dmodel}$ be the output of Multi-head Masked Attention layer of one of the decoder layers. What will be the size of the attention matrix in the Multi-head Cross Attention layer of the same decoder layer? Assume $d_q = d_v = d_k = d$ if required.

Attention matrix = $\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$



Size of W_K, W_V & W_K ?

Size of Attention matrix will be $T_2 \times T_1$

$W_q \rightarrow d_{model} \times d_q$.

than $Q \rightarrow Z_{T2 \times d_{model}} \cdot W_q \; d_{model} \times d_q \rightarrow Q \; T_2 \times d$

$K \rightarrow R_{T_1 \times d_{model}} \cdot W_k \; d_{model} \times d_k \rightarrow K_{T_1 \times d}$.

$V \rightarrow V_{T_1 \times d}$

$\therefore$ Score : $Q \cdot K^T \rightarrow T_2 \times T_1$ & Softmax (Score) → Attention Matrix

& $\therefore X = Score \cdot V^T \rightarrow T_2 \times T_1$

As per this, dimension of $W_q, W_k \& W_v$ should be $d_{model} \times d$.

$\therefore$ dim. of attention matrix is $T_2 \times T_1$

Suppose we run the pre-trained GPT model in an autoregressive fashion for generating a text sequence. Then, the statement that " We do not need to add the positional embedding to the predicted tokens at every time step" is

In GPT

Why do we add "positional Encoding" → To get context of position also.

In "auto regressive" also, we need to feed that info.

So answer is "True".

Suppose we have a dataset for machine translation tasks with thousands of samples. Suppose a team considers training the transformer model. The model could be trained in two approaches

A Autoregressive training

B Teacher forcing

Teacher forcing allows to converge faster.

Choose the correct statements

Options :

6406532475231. ✗ Approach *A* helps the model to converge faster than approach *B* False

6406532475232. ✓ Approach *B* helps the model converge faster than approach *A* ✓

6406532475233. ✓ One can start the training with approach *B* first and then switch to approach *A* after some training steps ✓

6406532475234. ✗ Once the training starts with approach *B* and then switching to approach *A* after some training steps can not be done false.

A team trained a Language model using the GPT architecture using a large corpus of text. The model configurations are given below

- Vocabulary size = 30000
- $dmodel = 512$
- length of context window $T = 1024$
- $dff = 4 * dmodel$
- $dq = dk = dv = 64$

They trained the model for millions of steps and stopped the training after the loss had decreased significantly. Then, they used the model to generate the text by giving the special starting token [BOS] as an input to the model. Select the length of the text sequence that the model could generate.

Options :

6406532475235. ✔ 512

6406532475236. ✔ 1024

6406532475237. ✔ 1536

6406532475238. ✔ 2048

Can generate any no. of tokens. till <End> or some other Condition (max. no. of tokens)

Choose the correct statements

Options :

6406532475245. ✔ All the parameters of the GPT model are randomly initialized during pre-training Yes

6406532475246. ✖ The model minimizes the CLM objective during fine-tuning to improve the performance

No, maximises.

6406532475247. ✖ All parameters of the GPT model are randomly initialized during fine-tuning

No. (

6406532475248. ✔ In general, fine-tuning requires a dataset with labels ✓yes.

Consider a GPT model used for Causal language modelling. We feed the input sentence "This is a cool idea" to the model by appending special staring [BOS] and ending [EOS] tokens (that is, "[BOS] This is a cool idea [EOS]"). Assume the context length of the model is 7. The attention matrix computed in one of the attention layers is given below

	BOS	This	is	a	cool	idea	<End>
0	1	0	0	0	0	0	0
1	0.5	0.5	0	0	0	0	0
2	0.3	0.2	0.5	0	0	0	0
3	0.1	0.1	0.8	0	0	0	0
4	0	0.15	0.1	0.6	0.25	0	0
5	0.25	0.35	0	0.2	0.1	0.1	0
6	0.1	0.1	0.1	0.1	0.1	0.4	0.1

Table 1: Attention score

The attention score matrix given in Table 1 is appropriate for the causal language modelling task

yes ∵ it is "masking" the info from future tokens.

Options :

6406532474526. ✔ True

6406532474527. ✖ False

6406532474528. ✖ Insufficient information

Assume the time step starts from $t = 0$ and ends at $t = 6$. Suppose the model is at time step $t = 4$, what is the attention value assigned for the word "is"?

at t=4

Response Type : Numeric

Evaluation Required For SA : Yes

Show Word Count : Yes

Answers Type : Equal

Text Areas : PlainText

Possible Answers :

0

At t=4, attention value for 'is' is 0.

Key Concept: GPT's Autoregressive Generation

GPT models generate text in an **autoregressive** manner, which means they predict the next token one at a time, conditioned on the previously generated tokens. This process continues until a predefined stopping condition (like reaching a maximum sequence length or a special token) is met.

Context Window $T = 1024$:

- The context window size $T = 1024$ defines the maximum number of tokens that the model can handle **at once** during training or inference.
- When generating sequences longer than 1024 tokens, the model uses a **sliding window** technique, where newly predicted tokens are fed back into the model, and the oldest tokens are dropped from the context.

Which of the following decoding strategies is (are) appropriate for machine translation tasks? Assume we have infinite compute and the values for $k \geq 3, p > 0.35$ where required.

Options :

6406532475249. ✖ Top-k sampling

6406532475250. ✔ Beam search with the beam size $K = 2$

6406532475251. ✖ Top-p sampling

6406532475252. ✔ Exhaustive search

6406532475253. ✖ None of these

Machine Translation does not require "Uncertainty" so it has to follow "deterministic strategy" So either. Exhaustive/Greedy/Beam Search.

Question Label: Multiple Choice Question

The statement that "the Next Sentence Prediction (NSP) task requires the BERT model to run autoregressively given the first sentence(or segment)" is

Options :

6406532475254. ✗ TRUE

6406532475255. ✓ FALSE

"BERT" model works on "bidirectional" context and does not have "auto regression".